Singly Linked List – Delete At End –Space Complexity and Algorithm

Program:

```
void deleteFromEnd(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        Node *previous = nullptr;

        while (current->next != nullptr)
        {
            previous = current;
            current = current->next;
        }
        if (previous == nullptr)
        {
            *head = nullptr;
        }
        else
        {
            previous->next = nullptr;
        }
        cout << "Deleted node: " << current->data << endl;
        free(current);
        current = nullptr;
        previous = nullptr;
        cout << "Node is deleted successfully." << endl;
    }
```

```
}

…..

 case 7:
        {
            cout << endl;
            cout << endl;
            deleteFromEnd(&head);
            break;
        }
```

Space Complexity Analysis

1. *Input Space*:

A. Parameter – Only Definition:

The input space consists of the parameters passed to the function, which in this case is a pointer to a pointer (Node ** head). This is a constant space allocation, so it contributes O(1) to the space complexity.

This view only considers the space taken by the function's parameters. The single parameter `Node **head` is a pointer-to-a-pointer, which has a fixed, constant size. Therefore, the input space complexity is **O(1)**.

- **One parameter (head, a double pointer)**
- **Size is a single machine word (plus possible alignment)**

  ⇒ Θ(1) = O(1)

B. Full – Data Definition:

This more common definition includes the entire data structure accessible from the parameters. If the linked list has **n nodes** before the deletion, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same single word for head (Θ(1))

• Plus every node that can be reached from *head before the call executes.

• If the list currently contains n nodes, those nodes together occupy Θ(n) bytes. ⇒ Θ(n) = O(n)

2. *Auxiliary Space*:

Auxiliary space refers to any additional space used by the function that is not part of the input. In this function, the auxiliary space is primarily used for:

- ***Temporary variables: current and previous, both of which are pointers to Node. These variables are used to traverse and modify the list.***

Since these are constant space variables that do not depend on the size of the input, they contribute O(1) to the auxiliary space.

Auxiliary space is the extra memory an algorithm uses during its execution, not counting the input.

1. `Node *current` **and** `Node *previous`: The function declares two pointers to traverse the list. Each pointer occupies a constant amount of memory.
2. **Iteration**: The `while` loop reuses these two pointers to find the end of the list. No new memory is allocated during the traversal. This affects the function's **time complexity** (making it O(n)), but not its space complexity.
3. `free(current)`: This function *releases* memory back to the system; it does not consume extra working space for the algorithm.

Since the function only needs a fixed number of extra variables (two pointers) to do its job, the amount of extra memory used is constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

• Two local pointers: current and previous → constant space
• A few implicit temporaries generated by the compiler → constant space
• No additional dynamic allocations (the function actually frees memory)
• No recursion → no call-stack blowup ⇒ Θ(1) = O(1)

Algorithm

$DELEND(START)$

1. $[LIST\ is\ Empty?]$

$If\ START = NULL, then:$

$Write:$ ``Error: List is empty.``

$Return\ and\ EXIT.$

$[DELETE\ NODE]$

$ELSE$

$SET\ CURRPTR := START\ [CURRPTR\ is\ a\ pointer\ to\ store\ address\ of\ last\ node].$

$SET\ PREVPTR := NULL\ [PREVPTR\ is\ a\ pointer\ to\ store\ address\ of\ previous\ node\ of\ last\ node].$

$[WHEN\ LAST\ HAS\ NODES\ GREATER\ THAN\ ONE]$

$REPEAT\ while\ LINK[CURRPTR] \neq NULL\ then:$

$SET\ PREVPTR := CURRPTR\ \left[\begin{array}{c}This\ will\ help\ PREVPTR\ pointer\ to\ hold\ address\ of\ previous\ node\ of \\ last\ node.\end{array}\right]$

$SET\ CURRPTR := LINK[CURRPTR]\left[\begin{array}{c}This\ will\ help\ CURRPTR\ pointer\ to\ hold\ address\ of \\ last\ node.\end{array}\right]$

$[END\ OF\ LOOP]$

$[WHEN\ FIRST\ NODE\ IS\ LAST\ NODE\ (OR)\ ONE\ NODE\ IN\ THE\ LIST,\quad THEN\ PREVPTR\ REMAINS\ NULL]$

$If\ PREVPTR = NULL\ , then:$

$Set\ START := NULL\ [This\ will\ delink\ HEAD\ pointer\ from\ First\ Node\ ]$

$[WHEN\ FIRST\ NODE\ IS\ NOT\ LAST\ NODE\ (OR)\ THERE\ IS\ MORE\ THAN\ ONE\ NODE, THEN\ PREVPTR\ IS\ NOT\ NULL]$

$ELSE$

$SET\ LINK[\ PREVPTR] := NULL\ \left[\begin{array}{c}This\ will\ delink\ previous\ node\ from\ last\ node\ and \\ make\ previous\ node\ the\ last\ node.\end{array}\right]$

$[END\ OF\ IF\ STRUCTURE]$

$Write:\ "Deleted\ node:\ "\ followed\ by\ INFO[CURRPTR]\ [Print\ the\ data\ of\ the\ node\ being\ deleted.]$

$FREE(CURRPTR)[Deallocate\ the\ memory\ used\ by\ the\ node.]$

$SET\ CURRPTR := NULL\ [Sets\ Dangling\ pointer\ CURRPTR\ to\ NULL]$

$SET\ PREVPTR := NULL\ [Sets\ Dangling\ pointer\ PREVPTR\ to\ NULL]$

$Write:\ "Node\ is\ deleted\ successfully."\ [Write\ Statement\ on\ Confirmation\ of\ Deletion]$

$[END\ OF\ IF\ STRUCTURE]$